

SPGCL: Subgraph Pattern-Aware Graph Contrastive Learning for High-Order Structural Representation

Zhe Zhan
Fudan University
Shanghai, China
zhazhe0514@gmail.com

Xiangshi Li
Fudan University
Shanghai, China
lixiangshi49@gmail.com

Tingting Wang
Dalian University of Technology
Dalian, China
ttwang93@outlook.com

Shuo Yu*
Dalian University of Technology
Dalian, China
shuo.yu@ieee.org

Henan Lei
Fudan University
Shanghai, China
leihen24@gmail.com

Abstract

Graph contrastive learning has emerged as a promising self-supervised approach for node representation learning, reducing reliance on human annotations. However, its limitation for modeling high-order structures leads to: loss of critical edges and nodes, diminished discriminability for higher-order semantics, and indistinguishable negative samples. To address these limitations, we propose SPGCL, a Subgraph Pattern-Aware Graph Contrastive Learning for Structural Representation. SPGCL enables the model to capture high-order structures by leveraging subgraph patterns. Specifically, SPGCL introduces subgraph patterns to differentiate high-order structures and preserve edges crucial for distinguishing these structures. In addition, it internalizes the subgraph pattern features into SP edge features, SP node features, and SP adjacency matrices to provide a more comprehensive structural representation. Furthermore, it utilizes subgraph pattern similarity and distance similarity, and restructures graph contrastive loss to sharpen negative-sample discrimination. Experiments on six real-world datasets demonstrate that SPGCL significantly outperforms state-of-the-art baselines.

CCS Concepts

• Computing methodologies → Semantic networks.

Keywords

Subgraph Pattern, Graph Contrastive Learning

ACM Reference Format:

Zhe Zhan, Xiangshi Li, Tingting Wang, Shuo Yu, and Henan Lei. 2026. SPGCL: Subgraph Pattern-Aware Graph Contrastive Learning for High-Order Structural Representation. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792522>

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792522>

1 Introduction

Graph contrastive learning (GCL) is a self-supervised graph representation learning technique. Most GCL methods [37, 40] can be delineated into three components, stochastic augmentation, embedding network, and contrastive learning. Stochastic augmentation perturbs graph structures to generate diverse views. The embedding network projects the augmented graph data into a latent space. Contrastive learning aims to maximize similarity between positive pairs while minimizing similarity between negative pairs. GCL has demonstrated its effectiveness and excels in downstream tasks such as node classification [35], node clustering [34], and link prediction [28].

However, due to the neighborhood propagation mechanism and the limitation on the number of embedding models' layers, GCL lacks sufficient capability to capture high-order structural information [20]. To address this limitation, researchers have turned to subgraph patterns, which are patterns of interconnections occurring in complex networks with higher frequency in randomized networks [24]. These subgraph patterns contain rich structural information and provide more nuanced insight into the underlying network structure. Building on this idea, MCN [17] integrates high-order structural motifs with graph attention mechanisms to capture higher-order interactions. Similarly, InfoMotif [26] regularizes the node representations of GNNs by maximizing the covariation information of node attributes across different network patterns. Furthermore, [28] captures regularity subgraph patterns by employing a motif-aware Riemannian generative-contrastive learning framework. Recently, MESERS [22] identifies subgraph patterns in global session graphs, encoding them as supernodes to enrich graph topology background. In addition, SKES [9] enriches node representations with structural knowledge and quantifies importance by measuring disparity against an uninformative reference. For temporal modeling, Motif-GNN [2] firstly leverages higher-order temporal motifs within the graph structure, utilizing a motif-wise multi-view attention module to weight informative motifs. Despite the promising performance of the above methods, most models employ subgraph patterns in a superficial manner, treating them merely as a modular component without full integration into the learning framework. Consequently, it will result in loss of critical edges and nodes, diminished discriminability for high-order semantics, and indistinguishable negative samples. In the following, we conduct a detailed analysis of these three deficiencies in Figure 1.

Prevailing stochastic augmentation strategies in GCL compromises critical topological structures and disrupt the intrinsic distribution of node attributes. Specifically, random removal of edges risks discarding vital connections, such as those between highly central nodes. The loss of such pivotal edges hinders effective neighbor aggregation for these central nodes, which are crucial for capturing global structural information. Furthermore, this stochastic edge deletion can inadvertently sever the communication pathways between densely connected communities, impeding essential message passing across distinct topological regions. Concurrently, the common practice of randomly masking node attributes to zero induces a distributional shift in the original feature space, obscuring the authentic characteristics of affected nodes. These limitations collectively underscore the following challenge: **How to preserve graph topology and feature distribution while generating effective contrastive views for GCL?**

Standard Graph Neural Networks (GNNs) exhibit limited discriminative capacity in capturing high-order graph structural patterns. In academic and social networks, open triads govern information flow dynamics [12], acting as bridges that mediate social interactions [3]. Conversely, closed triads emphasize influence propagation, trust formation [21], and cooperative behavior [8]. Discriminating between these motifs is crucial for GCL to encode higher-order semantic information. However, conventional GNNs fail to distinguish such patterns. When processing open and closed triads through single layer GNN modules, the learned representations exhibit negligible differentiation due to neighbor propagation constraints. This limitation persists in deep GNNs for capturing higher-order patterns. The increased layer stacking may capture broader structural contexts but inevitably induces feature homogenization (over-smoothing). Consequently, a fundamental challenge emerges: **How to enhance high-order structural discriminability without incurring over-smoothing?**

Current GCL frameworks fail to adequately model the intrinsic heterogeneity among negative samples. Prevailing methods uniformly treat all negative pairs, disregarding critical structural and topological relationships that govern semantic similarity. Specifically, two key dimensions of variation are overlooked: 1. Structural equivalence: Nodes exhibiting isomorphic local substructures should manifest closer representation proximity despite being negative pairs. Existing approaches fail to leverage such structural regularities, erroneously pushing representations of isomorphic nodes apart with equal penalty. 2. Multi-hop neighborhood affinity: Topologically proximate nodes inherently share greater contextual similarity than distant nodes. Current negative sampling strategies neglect this hierarchical proximity, imposing uniform separation constraints irrespective of underlying graph distance. Thus, structurally equivalent nodes are incorrectly distanced further than non-equivalent counterparts. In addition, neighboring nodes lack appropriate relative affinity compared to remote nodes. This homogenization of negative pairs discards discriminative information essential for capturing fine-grained graph semantics. We thus pose the core challenge: **How to design discrimination-aware negative sampling that preserves structural and neighborhood affinity?**

To address the above three questions, we propose a novel Subgraph Pattern-aware Graph Contrastive Learning named SPGCL.

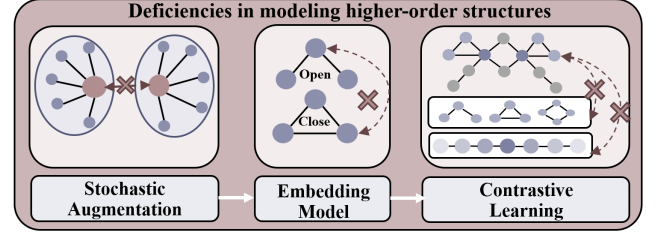


Figure 1: Deficiencies in modeling higher-order structures.

Specifically, we leverage subgraph pattern to reconstruct the three components of current GCLs, which are SP-augmentation, SP-GNNs and SP-CL. For SP-augmentation, we employ Gaussian blur rather than masking to zero on the raw data to prevent the distributional shift in the original feature space. We calculate the SP semantic significance of edges by SP frequency. Guided by SP semantic significance, the graph augmentation is conducted to preserve critical edges. For SP-GNNs, we design a subgraph pattern based message aggregation network. To simultaneously leverage multi-SP features and prevent over-smoothing, we employ a multilayer cascaded GNN network with residual structures [6]. For SP-CL, we utilize subgraph pattern to enhance the distinctiveness between negative samples, including subgraph pattern similarity and distance similarity. Subgraph pattern similarity is used to evaluate the structural equivalence, and distance similarity is used to evaluate the multi-hop neighborhood affinity. Our contributions are as follows.

- We propose SPGCL, a novel Subgraph Pattern-aware Graph Contrastive Learning. Based on subgraph pattern, we reconstruct the traditional GCL model into SP-augmentation, SP-GNNs and SP-CL.
- We propose and address three issues in GCL stemming from inadequate capture of subgraph pattern. Separately, we address the loss of central nodes and critical edges in graph augmentation, overcome the deficiency in capturing high-order graph structure features in graph representation learning, and compensate for the insufficiency in discriminating negative samples.
- Extensive experiments demonstrate that our learned representation is consistently competitive on classification, clustering, and link prediction tasks.

2 Related Work

2.1 Graph Representation Learning

Graph representation learning aims at capturing various properties of graphs in vector space [43]. Graph Convolutional Network (GCN) [16] represents the most paradigmatic approach for graph representation learning, which updates node embeddings by aggregating neighborhood information. The layer-wise propagation of the GCN can be formulated as:

$$\mathbf{H}^{(l+1)} = \sigma \left(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W}^{(l)} \right), \quad (1)$$

where $\mathbf{H}^{(l+1)}$ is the node feature matrix at the l_{th} layer, $\tilde{\mathbf{A}}$ is the adjacency matrix with added self-loops, $\tilde{\mathbf{D}}$ is the degree matrix

of $\tilde{\mathbf{A}}, \mathbf{W}^{(l)}$ is the trainable weight matrix at the l_{th} layer, σ is an activation function.

Subsequently, a variety of Graph Neural Networks (GNNs) with distinctive characteristics have emerged, tailored to diverse downstream objectives. Inspired by evolution theory, EGNN [23] estimates a graph's connections from node features using different graph structure estimators, creating model parameter populations. In addition, PEARL [30] employs graph-structured electronic health records, to address the challenge of predicting disease risks within Non-IID data environments. Different from direct embeddings, SiGNN [5] uses SNN-generated binary spikes as activation signals to modulate feature propagation in GNNs, preserving rich representations. For spatial-temporal graph representations, ST-GMA [39] employs adaptive contrastive learning with robust data augmentations to discover reliable spatio-temporal correlations. Moreover, INF-GNN [44] tackles data drift by distilling diversified invariant patterns via Relation Importance-based subgraphs and an informative temporal memory buffer.

2.2 Graph Contrastive Learning

Graph Contrastive Learning is a framework designed to learn unsupervised representations by maximizing the agreement between different augmented views. The core idea is to leverage data augmentation to create correlated views and use a contrastive loss function to enforce consistency between these views [41]. GRACE [46] firstly generates two stochastic contexts by randomly dropping edges and masking features and optimize the loss function between positive and negative samples. The contrastive-loss paradigm may be summarized as:

$$\mathcal{L}_{GCL} = -\frac{1}{|\mathcal{V}|} \sum_{i \in \mathcal{V}} \log \frac{\exp(\mathbf{h}_i^{(1)\top} \mathbf{h}_i^{(2)} / \tau)}{\exp(\mathbf{h}_i^{(1)\top} \mathbf{h}_i^{(2)} / \tau) + \sum_{j \neq i} \exp(\mathbf{h}_i^{(1)\top} \mathbf{h}_j^{(1)} / \tau)}, \quad (2)$$

where $\mathbf{h}_i^{(1)}, \mathbf{h}_i^{(2)}$ are two representations of node i from two perturbed views, $\mathbf{h}_j^{(1)}$ acts as the negative samples. Building on this foundation, HomoGCL [19] leverages homophily to enhance representation learning by treating similar neighbors as positive samples, eliminating the need for data augmentation. Some researchers use GCL to explore spatio-temporal graphs. Sp^2 [13] integrates spatial and spectral views of graphs by a stable and scalable spectral encoder. Furthermore, MARIO [45] incorporates adversarial graph augmentation and information bottleneck principles to enhance the out-of-distribution generalization of graph contrastive learning. EPAGCL [38] leverages error passing rate to adaptively balance adding or dropping edges, addressing the instability of edge addition. These GCLs achieve great success. However, they do not simultaneously address the aforementioned three challenges.

3 Problem Definition

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{A}, \mathbf{X}, \mathbf{Y})$ denote a featured graph, where $\mathcal{V} = \{v_1, v_2, \dots, v_n\}$ is a set of nodes, and $\mathcal{E} = \{e_1, e_2, \dots, e_m\}$ is a set of edges between nodes. $|\mathcal{V}|$ denotes the number of set $\mathcal{V}$. $\mathbf{A} \in \{0, 1\}^{|\mathcal{V}| \times |\mathcal{V}|}$ denotes adjacency matrix, $\mathbf{X} \in \mathbb{R}^{|\mathcal{V}| \times F}$ denotes the node feature, F denotes the dimensionality of the feature $\mathbf{X}$. $\mathbf{Y} \in \{0, 1\}^{|\mathcal{V}| \times C}$ denotes the node label. We define the projection

function $\phi_x(v_i)$ to find the node feature x_i and function $\phi_y(v_i)$ to find the node label y_i of node v_i . Let a graph $\mathcal{P} = (\mathcal{V}', \mathcal{E}', \mathbf{X}', \mathbf{Y}')$ is a subgraph pattern of a graph $\mathcal{G}(\mathcal{V}, \mathcal{E}, \mathbf{X}, \mathbf{Y})$, if $\mathcal{V}' \subset \mathcal{V}$, $\mathcal{E}' \subset \mathcal{E}$, $\phi_x(\mathcal{V}') = \mathbf{X}'$, $\phi_y(\mathcal{V}') = \mathbf{Y}'$, and $\mathcal{P}$ is a connected graph. We enumerate five kinds of subgraph patterns ranging from third to fourth order, separately denoted as follows: $\mathcal{P} = \{P_1, P_2, P_3, P_4, P_5\}$. Based on $\mathcal{P}$, we define five subgraph pattern adjacency matrices as $\mathbf{A}_P = \{\mathbf{A}_{P_1}, \mathbf{A}_{P_2}, \mathbf{A}_{P_3}, \mathbf{A}_{P_4}, \mathbf{A}_{P_5}\}$. The original graph adjacency $\mathbf{A}$ represents the connection between two nodes. Similarly, $\mathbf{A}_{P_i}$ denotes the connection strength between two nodes based on P_i . Specifically, the value in the m_{th} row and n_{th} column of $\mathbf{A}_{P_i}$ represents the times that $\mathbf{V}_m$ and $\mathbf{V}_n$ co-occur within P_i . Our goal is to learn a representation vector $\mathbf{Z}$ for each node in graph through self-supervised graph contrastive learning. $\mathbf{Z}$ can be used in different types of downstream tasks, such as node classification, node clustering, and link prediction. The notations are provided in the Appendix.

4 The Proposed Model

4.1 Overview

The comprehensive SPGCL framework is visually outlined in Figure 2. SPGCL comprises three components, SP-Augmentation, SP-GNN, and SP-CL. The details are introduced as follows.

4.2 Subgraph Pattern Augmentation

We initially design a SP Extractor to extract the SP feature from the network. Subsequently, we refine the extracted SP feature to better align with the requirements of graph augmentation. Ultimately, we integrate the SP feature into the graph augmentation process. The details are shown as follows.

Inspired by [10, 36], we map the occurrence frequencies of nodes and edges to SP node features and SP edge features. As shown in Figure 2, the SP-Extractor is designed to respectively uncover the SP node feature in attribute augmentation and SP edge feature in structure augmentation. For attribute augmentation, we count the subgraph patterns class number contained in each node and translate the statistics results to a matrix, as SP node feature $\mathbf{V} \in \mathbb{R}^{|\mathcal{V}| \times t}$, where t denotes the number of subgraph patterns class, $|\mathcal{V}|$ demotes the number of nodes in $\mathcal{V}$, $\mathbf{V}_{i,j}$ denotes the number of subgraph patterns containing node i and belonging to j class. Next, we will utilize $\mathbf{V}$ in attribute augmentation and structure augmentation. We splice the SP-features $\mathbf{V}$ and the original attributes $\mathbf{X} \in \mathbb{R}^{|\mathcal{V}| \times F}$ to obtain the SP-pertaining attributes $\tilde{\mathbf{X}}$:

$$\tilde{\mathbf{X}} = [\mathbf{X}, \mathbf{V}]. \quad (3)$$

Rather than masking node attributes [31], we enhance the attribute injecting with Gaussian noise to safeguard the initial attribute distribution:

$$\hat{\mathbf{X}} = \tilde{\mathbf{X}} + \epsilon, \quad (4)$$

where ϵ is a random noise drawn from Gaussian distribution $\mathcal{N}(0, \sigma^2)$, and σ is the standard deviation of the noise, and $\hat{\mathbf{X}} = \{x_1, x_2, \dots, x_n\}$. x_i denotes the augmented attribute of node v_i .

For structure augmentation, we count the subgraph patterns class number contained in each edge and translate the statistics results to a matrix, as SP edge feature $\mathbf{E} \in \mathbb{R}^{|\mathcal{E}| \times t}$, where t denotes the number of subgraph patterns, $\mathbf{E}_{i,j}$ denotes the number of subgraph

In order to more effectively leverage the captured subgraph pattern adjacency matrix, we have designed a residual network to enhance model depth and capture long-range information. We integrate the output $\mathbf{H}^1$ of the initial layer with the input of each subsequent layer through residual connections to extend the depth of GNN layer:

$$\mathbf{H}^{i+1} = \text{GAT}(\mathbf{A}'_P, \mathbf{H}^i + \mathbf{H}^1), \forall i \in \{1, 2, 3, 4, 5\} \quad (14)$$

After we obtain the output of all layers, we perform a weighted summation of the outputs from each layer to obtain the final representation $\mathbf{Z}$:

$$\mathbf{Z} = \sum_{i \in \{1, 2, \dots, 6\}} \mathbf{W}^i \mathbf{H}^i. \quad (15)$$

4.4 Subgraph Pattern CL

In this section, we initially delve into the conventional graph contrastive loss. Subsequently, we modify the contrastive loss through the incorporation of subgraph patterns.

In details, to train the graph encoder in an unsupervised manner that can enrich the semantic information, we use a contrast loss to maximize the mutual information between the same nodes in different views. For node v_i , the node embedding in the two views is denoted as u_i and v_i . We call v_i as anchor. (u_i, v_i) represents a positive sample pair. The contrastive loss is as follows:

$$\mathcal{D}(u_i, v_i) = \log \frac{e^{\frac{\delta(u_i, v_i)}{\tau}}}{\sum_{k=1}^N e^{\delta(u_i, v_k)/\tau} + \sum_{k=1, k \neq i}^N e^{\delta(u_i, u_k)/\tau}}, \quad (16)$$

where $\delta(u_*, v_*) = \mu(\mathbf{g}_Y(u_*), \mathbf{g}_Y(v_*))$, τ denotes the temperature parameter, which controls the model's discrimination against negative samples, and $\mathbf{g}(\cdot)$ denotes the embedding models. $\mu(\cdot)$ denotes the similarity measurement function, for example, cosine similarity, vector product, Euclidean distance, etc. Here, to simplify computational complexity, we employ the vector product. For the selection of negative samples, we treat all nodes except anchor as negative samples. Since the two views are symmetrical, the final loss function of the model can be defined as

$$\mathcal{J} = \frac{1}{2N} \sum_{i=1}^N [\mathcal{D}(u_i, v_i) + \mathcal{D}(v_i, u_i)]. \quad (17)$$

However, we identified a shortcoming in the aforementioned loss functions, which is their equal treatment of repelling each negative sample from the positive sample. We improve this situation in two directions: subgraph pattern similarity and distance similarity. For subgraph pattern similarity, it is because the nodes with similar subgraph pattern should be given more similarity in attributes. Thus, we calculate the subgraph pattern similarity matrix by SP node feature $\mathbf{V}$:

$$\mathbf{M}_{SP} = \mathbf{V}^\top \mathbf{V}, \quad (18)$$

where the i_{th} row and j_{th} column in $\mathbf{M}_{SP}$ denotes the subgraph pattern similarity of node v_i and node v_j . For distance similarity, the reason is the nodes with long distance deserve less similarity. We calculate the shortest distance matrix $\mathbf{M}_D$ between nodes by Breadth-First Search (BFS), in which the i_{th} row and j_{th} column denotes the distance between node v_i and node v_j . It should be noted that some nodes are unreachable. Thus, we utilize twice the maximum reachable distance as the initialization parameter to represent the distance between nodes that are not connected by a

path. Through the subgraph pattern similarity matrix and distance matrix, our new contrastive loss is as follows:

$$\mathcal{D}(u_i, v_i) = \log \frac{e^{\frac{\delta(u_i, v_i)}{\tau}}}{Neg}, \quad (19)$$

$$Neg = \sum_{k=1}^N \frac{e^{\delta(u_i, v_k)/\tau} \mathbf{M}_{SPik}}{\mathbf{M}_{Dik}} + \sum_{k=1, k \neq i}^N \frac{e^{\delta(u_i, u_k)/\tau} \mathbf{M}_{SPik}}{\mathbf{M}_{Dik}}. \quad (20)$$

5 Experiments

5.1 Datasets and Baselines

5.1.1 DataSets. We conduct experiments on six real-world datasets, which are Citationv1, DBLPv7, Acmv9, Polblogs, Amazon computer and Amazon photo. They can be categorized into academic network and social networks. For academic network, Citationv1 [29] is from from Microsfot Academic Atlas. DBLPv7 [29] is from DBLP Computer Science Bibliography. Acmv9 [29] is from Computer Machinery Association. For social network, Polblogs [1] is a network composed of hyperlinks between different pages on American Political Network. Amazon computer [27] and Amazon photo [11] are fragments of the Amazon co-purchase graph. The statistics of datasets are provided in the Appendix. Table 6 provides detailed statistical information for these six datasets. Where "Node", "Edge" and "Class" respectively represent the number of nodes, edges and node label categories. We choose the five most common subgraph patterns in social networks and transportation networks. And the last five rows represent the average number of occurrences of five kinds of subgraph patterns in each dataset.

5.1.2 Baselines. To fully validate the performance of SPGCL, it is compared with eight existing methods, containing both supervised method and unsupervised method. For supervised method, we choose GCN [16], SGC [42], GCNII [7] and GATv2 [4] as baseline methods. For unsupervised method, we choose GAE [15], GRACE [46], MVGRL [14], DGI [33], HomoGCL [19], MotifRGC [28] and MHGCL[47] as baseline methods.

5.1.3 Experimental Setup. In the context of unsupervised methods, a linear classifier is employed to predict node labels. The maximum epoch is restricted to 2,000, with a tolerance of 20 in place. 10% of the datasets is reserved for training the classifiers, while the remaining 90% serves as test datasets. In the case of supervised methods, we adhere to the original recommended hyperparameters. For each node type, a training set comprising 20 samples is randomly selected, while 500 samples constitute the validation set, and the rest samples make up the testing set. All experimental computations are carried out using Python 3.10 and the Pytorch framework.

5.2 Node Classification

We perform node classification experiments on six datasets. The results are introduced in Table 1. The "best" and "avg" represents the best-performing and average outcomes of the model. The "OOM" denotes out of memory. From the experimental results of node classification on five datasets, we can see that SPGCL outperform all self-supervised baselines or even the supervised ones. The node classification results reveal that SPGCL demonstrates the ability to both uncover latent patterns within academic networks and capture subgraph similarity in social networks. SPGCL is able to adaptively

Table 1: Node classification results on six realworld datasets. Optimal results are highlighted in boldface, while second-best results are indicated with an underline. "OOM" denotes out of memory.

Methods	Acmv9		Dblpv7		Citationv1		Computers		Photo		Polblogs	
	best	avg	best	avg	best	avg	best	avg	best	avg	best	avg
GCN	66.01	62.85	67.63	64.94	67.42	66.09	81.59	72.68	91.42	86.59	95.31	94.84
SGC	58.40	56.64	60.42	59.04	63.28	61.13	64.07	59.43	84.16	81.73	95.49	94.79
GCNII	72.33	70.18	73.08	71.66	74.42	71.66	63.65	62.46	70.15	62.38	95.52	94.97
GATv2	64.10	63.34	72.12	70.04	68.13	66.61	82.95	79.54	92.23	91.61	95.31	94.92
GAE	50.08	47.62	52.31	50.33	53.19	51.45	82.15	79.61	88.94	88.32	92.30	91.69
DGI	70.46	68.47	70.47	69.02	74.58	73.28	87.25	86.78	92.67	92.26	94.28	92.46
GRACE	70.72	69.17	70.40	67.90	74.63	73.18	77.78	72.35	85.01	82.21	91.00	89.40
MVGRL	71.19	69.79	72.74	70.58	77.25	76.18	87.50	87.19	91.26	90.03	92.52	90.61
HomoGCL	72.01	71.63	72.72	71.26	77.23	76.85	<u>88.63</u>	<u>88.41</u>	<u>93.03</u>	<u>92.76</u>	93.72	92.79
MotifRGC	79.05	78.44	73.07	72.69	78.3	77.51	OOM	OOM	OOM	OOM	<u>95.58</u>	<u>95.09</u>
MHGCL	<u>79.58</u>	<u>78.93</u>	<u>77.68</u>	<u>77.14</u>	<u>82.29</u>	<u>81.75</u>	83.24	82.83	91.89	91.50	93.87	93.06
SPGCL	80.59	80.12	77.85	77.24	83.73	83.24	88.73	88.48	93.09	92.83	95.61	95.39

fuse multi-order structural information, thereby perceiving complex semantics. For baselines, MHGCL excels on ACMv9, DBLPv7, and Citationv1 datasets by leveraging inherent academic network homophily. Its Hop Augmentation reinforces representation consistency among similar nodes through multi-hop propagation. Furthermore, HomoGCL performs well on Computer and Photo. We posit that adjacent shoppers exhibit heightened homophily, which HomoGCL effectively captures by treating them as positive samples. However, neither model achieves strong performance across all datasets, underscoring SPGCL's capability to adaptively select subgraph patterns for learning, which partially enhances its generalization capability.

5.3 Node Clustering

Table 2: Node clustering results on three realworld datasets. Optimal results are highlighted in boldface. Second-best results are indicated with an underline.

Methods	Acmv9		Dblpv7		Citationv1	
	best	avg	best	avg	best	avg
GAE	35.58	34.59	38.16	36.42	35.30	34.70
DGI	63.64	62.69	63.90	60.00	61.53	60.93
GRACE	56.93	56.54	50.91	50.12	58.98	57.24
MVGRL	<u>64.30</u>	<u>63.88</u>	<u>64.16</u>	<u>63.69</u>	<u>73.32</u>	<u>71.84</u>
HomoGCL	56.21	55.43	62.83	62.19	54.09	53.21
MotifRGC	63.62	63.30	63.77	62.51	66.64	66.11
MHGCL	62.31	61.82	58.02	57.57	70.41	70.04
SPGCL	66.40	64.42	66.64	65.86	73.76	71.96

For the node clustering, we conduct experiments using an unsupervised approach on three acmv9, dblpv7 and citationv1. We obtain node embeddings from the original graph data through SP-GCN. The k-means algorithm is then applied to the embeddings for clustering. Due to the fact that the choice of initial nodes has a limited impact on the k-means results, clustering outcomes tend to be unstable. To address this issue, we execute the node clustering task 20 times and compute the average of the best three results.

Table 3: Link prediction results on three realworld datasets. Optimal results are highlighted in boldface. Second-best results are indicated with an underline. .

Methods	Acmv9		Dblpv7		Citationv1	
	best	avg	best	avg	best	avg
GAE	98.74	98.50	96.97	96.62	95.52	94.79
DGI	94.76	88.62	88.18	83.63	91.32	78.37
GRACE	98.50	98.04	97.17	95.19	97.34	96.29
MVGRL	98.00	85.76	73.48	70.08	94.88	86.39
HomoGCL	<u>99.20</u>	<u>98.79</u>	98.45	98.21	98.35	98.20
MotifRGC	99.09	98.22	<u>98.81</u>	<u>98.27</u>	<u>98.41</u>	<u>98.25</u>
MHGCL	99.12	98.97	97.96	97.78	98.34	98.02
SPGCL	99.25	98.89	98.97	98.31	98.54	98.43

We consider the best three results to have appropriate cluster centers at the beginning. The clustering accuracy results of SPGCL are presented in Table 2. When compared to the best-performing unsupervised method MVGRL, SPGCL has demonstrated a 1% to 2% improvement on the acmv9 and dblpv7. We posit that structurally simpler method like MVGRL can better preserve graph semantics, thereby enhancing clustering performance. This empirically validates that our subgraph pattern augmentation effectively prevents corruption of original graph semantics. Besides, SPGCL excels MotifRGC, HomoGCL and MHGCL, which means SPGCL can better extract high-order structures to identify the network center.

5.4 Link Prediction

For the link prediction task, we conduct experiments on Acmv9, Dblpv7 and Citationv1. We designate the edges that exist in the graph as positive edges, equal to the total number of edges in the graph. Additionally, to differentiate whether an edge truly exists in the graph, we randomly sample false edges between nodes within the graph. We generate a number of negative edges five times that of the positive samples. To compute similarity scores between nodes, we utilize the product of every two nodes. Subsequently, we predict

the presence of edges based on the higher score, while a lower score indicated the absence of edges. The link prediction results are presented in Table 3. Comparing SPGCL with all other unsupervised methods, it is evident that SPGCL has achieved state-of-the-art results in link prediction tasks. Additionally, the avg accuracy is very close to the best accuracy, indicating the remarkable stability of SPGCL across these three datasets. The reason is that SPGCL incorporates edge subgraph pattern information into node embeddings, enabling node embeddings to distinguish the presence of edges.

5.5 Ablation Study

Table 4: Ablation study results of SPGCL by node classification tasks.

Methods	Acmv9		Dblpv7		Citationv1	
	best	avg	best	avg	best	avg
w/o M_D	73.24	72.16	74.87	74.15	81.92	81.62
w/o M_{SP}	79.05	78.23	76.47	75.71	82.92	82.63
w/o ϵ	78.88	77.80	74.60	73.73	80.18	79.86
w/o RC	79.35	78.92	76.16	75.52	82.63	82.39
w/o V	79.58	79.10	75.99	75.33	82.84	82.47
w/o w_E	80.01	78.90	76.30	75.97	82.78	82.43
SPGCL	80.59	80.12	77.85	76.98	83.73	83.24

In this subsection, our aim is to investigate the influence of M_D , M_{SP} , ϵ , RC , V and w_E . For w/o M_{SP} and w/o M_D , we cancel their contributions separately in the loss function. For w/o ϵ , we do not enhance the node attribute by Gaussian noise. w/o RC means we eliminate the residual connections in SP-GNN. w/o V means that the matrix that reflects the number of node subgraph patterns is canceled. w/o w_E indicates that we use random edge drop strategies rather than SP-Weight. The node classification results are summarized in Table 4. We can deduce from the data that each component contributes to the model to a certain extent, among which the distance matrix has a significant impact on the experimental results. Our analysis suggests that the distance matrix enhances the model’s ability to perceive topological information of negative samples and enriches the semantic space of negative samples, thereby improving the model’s performance. Furthermore, Gaussian noise also contributes significantly to the model. We posit that the introduction of random Gaussian noise enhances the diversity among different nodes, which in turn augments the capacity for contrastive learning and elevates the model’s generalizability.

5.6 Edge Weights Statistics

As shown in Figure 3, we conducted a statistical analysis of the weight W_e first map the edge weights to the interval 0 to 100. The horizontal axis represents the range from 0 to 100. We then calculate the distribution of the dataset within the intervals to obtain the density distribution. The width of the instances represents the number of instances of edge weights in a certain interval. In brief, we can understand it as the density of weights. In Figure 3, the

original edge weight distributions are extremely uneven. The low-weight interval is characterized by a high density, whereas the mid-weight and high-weight intervals exhibit near-zero density. This pattern is indicative of a long-tailed distribution. This extreme distribution will cause most edges to have low weights, thereby resulting in excessive edge dropping. Meanwhile, a small number of edges will experience overheating, causing the model to overly focus on individual edges. Both of them have a negative impact on the model’s performance. The updated edges weight densities are shown in the blue part of each subgraph in Figure 3. There is no problem of a large number of low-weight edges or overheating of central edges. For Polblogs, the original distribution of edge weights is reasonable and does not need to be transformed into a normal distribution.

5.7 Dropping Rate Analysis

In this section, we analyze the upper bound u_e and the lower bound l_e . We conduct node classification experiments on the polblogs dataset, using varying upper and lower bounds as parameters for our analysis. As shown in Figure 4, we separately set the upper bound u_e from 0.1 to 0.5. Meanwhile, we set the upper bound l_e from 0 to 0.5. We construct a three-dimensional histogram to represent the interplay of these parameters, with the x axis denoting u_e , the y axis representing l_e , and the z axis illustrating the node classification results of the model on the polblog dataset. The x- and y-axes are both incremented by 0.1. The z-axis is scaled from 0.94 to 0.96 with an increment of 0.002. It is important to note that a subset of the z-axis data is based on the condition that u_e exceeds l_e . The aforementioned factors lead to the manifestation of several vacant areas within the visual depiction. The visual representation of the experimental data indicates that superior performance is observed under the condition that the lower bound is restricted between 0.1 and 0.2, and the upper bound is defined between 0.2 and 0.3. When both the lower and upper bounds are very small, it becomes challenging to eliminate edges that would otherwise provide sufficient divergence between two enhanced views, thereby leading to a degradation in the performance of the contrastive learning model. Conversely, when the bounds are set to large values, it may result in the loss of social structural associations for the nodes, thereby causing a loss of information.

5.8 Edge Subgraph Pattern Analysis

To further validate the effectiveness of our edge dropping strategy, we analyze the average edge subgraph patterns num before and after augmentation. Specifically, we compute the average number of subgraph patterns associated with each edge—denoted as “BeforeEG” (before edge dropping) and “AfterEG” (after edge dropping)—across three datasets: Acmv9, Dblpv7, and Citationv1.

As illustrated in the left subfigure of Figure 5, the AfterEG values consistently exceed the BeforeEG values across all datasets, with an average increase of approximately 0.05. This indicates that our edge dropping strategy preferentially retains edges that participate in a larger number of high-order subgraph patterns, thereby preserving structurally significant connections while discarding less informative ones.

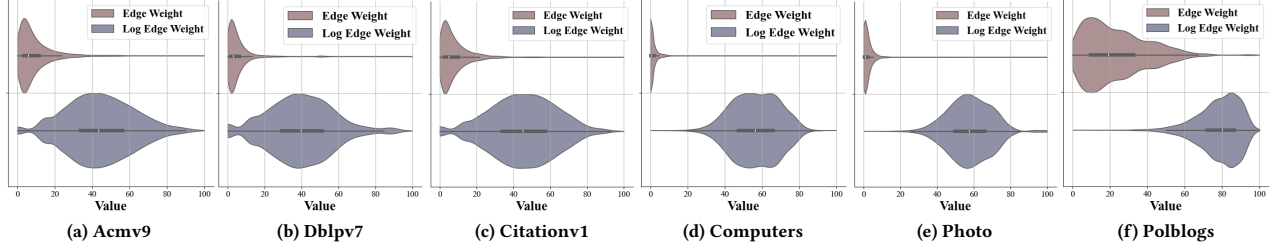


Figure 3: Overall description for all subfigures.

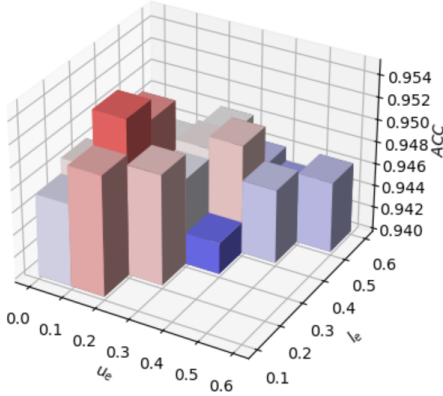
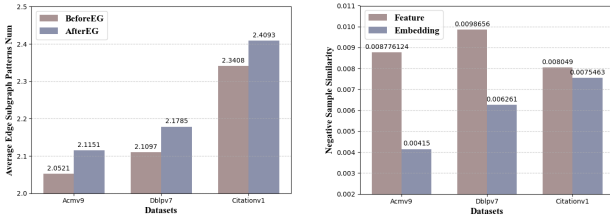
Figure 4: Dropping rate analysis of node classification from different upper bound u_e and lower bound l_e .

Figure 5: The effectiveness of our edge-dropping strategy. The comparison of the average edge subgraph pattern num before and after edge dropping. And the comparison of feature similarity and embedding similarity.

This result aligns with our design intuition: edges involved in more subgraph patterns are likely to play more critical roles in maintaining the global topology and semantic structure of the graph. By leveraging SP edge features to guide the augmentation process, SPGCL effectively enhances the structural representativeness of the augmented graphs, which in turn facilitates more informative contrastive learning.

5.9 Negative Sample Similarity Analysis

In this section, we calculate the similarity of negative samples before and after our model on three datasets. We randomly sample

1,000 negative sample nodes and calculate the cosine similarity between them, using the average value. As shown in the right subfigure of Figure 5, "Feature" denotes the original node feature, and "Embedding" means the embeddings that we get from our model. We can see that the similarity of negative sample embeddings is lower than that of their original node features. This demonstrates that our subgraph pattern and distance-aware contrastive loss effectively separates dissimilar nodes in the embedding space. By incorporating structural equivalence and neighborhood affinity, SPGCL avoids homogenizing negative samples and learns a more nuanced, semantically-rich representation space.

6 Conclusion

In this study, we propose a novel Subgraph Pattern-aware Graph Contrastive Learning, which addresses the issue of inadequate capturing of subgraph topological structures. Its uniqueness lies in adaptively constructing and encoding subgraphs based on the subgraph patterns. We enhance node attribute information by merging the nodes' subgraph patterns matrix with their original features. To harness the local topology information, we compute weightage for both nodes' and edges' subgraph patterns. These weights facilitate Gaussian noise node masking and dynamic weighted edge dropping, contributing to the richness of our method. The experimental evaluation of node classification, node clustering and link prediction on six benchmark datasets underscores SPGCL's superior performance. After the experiments, we conduct a detailed visual analysis of the results and hyperparameters. By adaptively exploring subgraph patterns, GCL are able to learn better graph structure features and to understand higher-order semantic information without introducing the three problems in introduction. However, SPGCL presents scalability limitations, exhibiting high computational complexity on large-scale graphs or hypergraphs. Future work will focus on addressing the challenge of scalability and computational efficiency in large-scale dynamic graph applications [18].

Acknowledgments

This work is supported by the National Natural Science Foundation of China (No. 62576075).

References

- [1] Lada A Adamic and Natalie Glance. 2005. The political blogosphere and the 2004 US election: divided they blog. In *Proceedings of the international workshop on Link discovery*. 36–43.
- [2] Nourhan Ahmed, Vijaya Krishna Yalavarthi, and Lars Schmidt-Thieme. 2025. Motif-aware Graph Neural Networks for Networked Time Series Imputation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Toby Walsh, Julie Shah, and Zico Kolter (Eds.). AAAI Press, 11409–11417. doi:10.1609/AAAI.V39I11.33241
- [3] Maciej Besta, Raphael Grob, Cesare Miglioli, Nicola Bernold, Grzegorz Kwasniewski, Gabriel Gjini, Raghavendra Kanakagiri, Saleh Ashkboos, Lukas Gianinazzi, Nikoli Dryden, et al. 2022. Motif prediction with graph neural networks. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 35–45.
- [4] Shaked Brody, Uri Alon, and Eran Yahav. 2022. How Attentive are Graph Attention Networks?. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net. <https://openreview.net/forum?id=F72ximsx7C1>
- [5] Dong Chen, Shuai Zheng, Muhao Xu, Zhenfeng Zhu, and Yao Zhao. 2025. SiGNN: A spike-induced graph neural network for dynamic graph representation learning. *Pattern Recognition* 158 (2025), 111026. doi:10.1016/j.patcog.2024.111026
- [6] Lei Chen, Le Wu, Richang Hong, Kun Zhang, and Meng Wang. 2020. Revisiting graph based collaborative filtering: A linear residual graph convolutional network approach. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 27–34.
- [7] Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. 2020. Simple and deep graph convolutional networks. In *International Conference on Machine Learning*. 1725–1735.
- [8] Xuexin Chen, Ruichu Cai, Yuan Fang, Min Wu, Zijian Li, and Zhifeng Hao. 2023. Motif graph neural network. *IEEE Transactions on Neural Networks and Learning Systems* (2023).
- [9] Yankai Chen, Yixiang Fang, Qiongyan Wang, Xin Cao, and Irwin King. 2024. Deep structural knowledge exploitation and synergy for estimating node importance value on heterogeneous information networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 8302–8310.
- [10] Yankai Chen, Taotao Wang, Yixiang Fang, and Yunyu Xiao. 2025. Semi-supervised node importance estimation with informative distribution modeling for uncertainty regularization. In *Proceedings of the ACM on Web Conference 2025*. 3108–3118.
- [11] Eli Chien, Jianhao Peng, Pan Li, and Olgica Milenkovic. 2020. Adaptive universal generalized pagerank graph neural network. *arXiv preprint arXiv:2006.07988* (2020).
- [12] Sarita S Deshpande, Graham A Smith, and Wim van Drongelen. 2023. Third-order motifs are sufficient to fully and uniquely characterize spatiotemporal neural network activity. *Scientific Reports* 13, 1 (2023), 238.
- [13] Simon Geisler, Arthur Kosmala, Daniel Herbst, and Stephan Günnemann. 2024. Spatio-Spectral Graph Neural Networks. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. 49022–49080. doi:10.52202/079017-1554
- [14] Kaveh Hassani and Amir Hosein Khasahmadi. 2020. Contrastive multi-view representation learning on graphs. In *International Conference on Machine Learning*. 4116–4126.
- [15] Thomas N Kipf and Max Welling. 2016. Variational graph auto-encoders. *arXiv preprint arXiv:1611.07308* (2016).
- [16] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*. OpenReview.net. <https://openreview.net/forum?id=SJU4ayYgl>
- [17] John Boaz Lee, Ryan A. Rossi, Xiangnan Kong, Sungchul Kim, Eunye Koh, and Anup Rao. 2019. Graph Convolutional Networks with Motif-based Attention. In *Proceedings of the ACM International Conference on Information and Knowledge Management (Beijing, China) (CIKM '19)*. Association for Computing Machinery, New York, NY, USA, 499–508. <https://doi.org/10.1145/3357384.3357880>
- [18] Dongyuan Li, Satoshi Kosugi, Ying Zhang, Manabu Okumura, Feng Xia, and Renhe Jiang. 2025. Revisiting dynamic graph clustering via matrix factorization. In *Proceedings of the ACM on Web Conference 2025*. 1342–1352.
- [19] Wen-Zhi Li, Chang-Dong Wang, Hui Xiong, and Jian-Huang Lai. 2023. HomoGCL: Rethinking Homophily in Graph Contrastive Learning. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, Ambuj K. Singh, Yizhou Sun, Leman Akoglu, Dimitrios Gunopulos, Xifeng Yan, Ravi Kumar, Fatma Özcan, and Jieping Ye (Eds.). ACM, 1341–1352. doi:10.1145/3580305.3599380
- [20] Xianxian Li, Jiayue Zeng, Zhigang Sun, and Li'e Wang. 2024. Higher-order Semantic-aware Adaptive Graph Contrastive Learning. In *International Joint Conference on Neural Networks*. 1–8.
- [21] Sungsu Lim and Jae-Gil Lee. 2016. Motif-based embedding for graph clustering. *Journal of Statistical Mechanics: Theory and Experiment* 2016, 12 (2016), 123401.
- [22] Ronghua Lin, Chang Liu, Hao Zhong, Chengzhe Yuan, Guohua Chen, Yuncheng Jiang, and Yong Tang. 2025. Motif and supernode-enhanced gated graph neural networks for session-based recommendation. *Neural Networks* 187 (2025), 107406. doi:10.1016/J.NEUNET.2025.107406
- [23] Zhaowei Liu, Dong Yang, Yingjie Wang, Mingjie Lu, and Ranran Li. 2023. EGNN: Graph structure learning based on evolutionary computation helps more in graph neural networks. *Applied Soft Computing* 135 (2023), 110040.
- [24] R. Milo, S. Shen-Orr, S. Itzkovitz, N. Kashtan, D. Chklovskii, and U. Alon. 2002. Network Motifs: Simple Building Blocks of Complex Networks. *Science* 298 (2002).
- [25] R. Sanders. 1987. The Pareto principle: its use and abuse. *Journal of Services Marketing* 1, 2 (1987), 37–40.
- [26] Aravind Sankar, Junting Wang, Adit Krishnan, and Hari Sundaram. 2020. Beyond Localized Graph Neural Networks: An Attributed Motif Regularization Framework. In *IEEE International Conference on Data Mining*. 472–481.
- [27] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Pitfalls of graph neural network evaluation. *arXiv preprint arXiv:1811.05868* (2018).
- [28] Li Sun, Zhenhao Huang, Zixi Wang, Feiyang Wang, Hao Peng, and Philip S. Yu. 2024. Motif-Aware Riemannian Graph Neural Network with Generative-Contrastive Learning. *Proceedings of the AAAI Conference on Artificial Intelligence* 38, 8 (Mar. 2024), 9044–9052. <https://ojs.aaai.org/index.php/AAAI/article/view/28754>
- [29] Jie Tang, Jing Zhang, Limin Yao, Juanzi Li, Li Zhang, and Zhong Su. 2008. Arnetminer: extraction and mining of academic social networks. In *Proceedings of the ACM SIGKDD international conference on Knowledge discovery and data mining*. 990–998.
- [30] Tao Tang, Zhuoyang Han, Zhen Cai, Shuo Yu, Xiaokang Zhou, Taiwo Oseni, and Sajal K. Das. 2024. Personalized Federated Graph Learning on Non-IID Electronic Health Records. *IEEE Trans. Neural Networks Learn. Syst.* 35, 9 (2024), 11843–11856. doi:10.1109/TNNLS.2024.3370297
- [31] Yijun Tian, Kaiwen Dong, Chunhui Zhang, Chuxu Zhang, and Nitesh V Chawla. 2023. Heterogeneous graph masked autoencoders. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 9997–10005.
- [32] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations*. OpenReview.net. <https://openreview.net/forum?id=rjXmpikCZ>
- [33] Petar Velickovic, William Fedus, William L. Hamilton, Pietro Liò, Yoshua Bengio, and R. Devon Hjelm. 2019. Deep Graph Infomax. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net. <https://openreview.net/forum?id=rklz9iAcKQ>
- [34] Chenhao Wang, Yong Liu, Yan Yang, and Wei Li. 2024. HeterGCL: graph contrastive learning framework on heterophilic graph. In *Proceedings of the International Joint Conference on Artificial Intelligence*. 2397–2405.
- [35] Teng Xiao, Huaisheng Zhu, Zhengyu Chen, and Suhang Wang. 2024. Simple and asymmetric graph contrastive learning without augmentations. *Advances in Neural Information Processing Systems* 36 (2024).
- [36] Jin Xu, Shuo Yu, Ke Sun, Jing Ren, Ivan Lee, Shirui Pan, and Feng Xia. 2020. Multivariate relations aggregation learning in social networks. In *Proceedings of the ACM/IEEE Joint Conference on Digital Libraries*. 77–86.
- [37] Yanchen Xu, Siqi Huang, Hongyuan Zhang, and Xuelong Li. 2025. Why does dropping edges usually outperform adding edges in graph contrastive learning?. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 21824–21832.
- [38] Yanchen Xu, Siqi Huang, Hongyuan Zhang, and Xuelong Li. 2025. Why Does Dropping Edges Usually Outperform Adding Edges in Graph Contrastive Learning? *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 20 (Apr. 2025), 21824–21832. doi:10.1609/aaai.v39i20.35488
- [39] Hanchen Yang, Jiannong Cao, Wengen Li, Yu Yang, Xiaoyi Li, Lingbai Kong, Yichao Zhang, Jihong Guan, and Shuigeng Zhou. 2025. Towards Robust and Interpretable Spatial-Temporal Graph Modeling for Traffic Prediction. *ACM Transactions on Knowledge Discovery from Data* 19, 9 (2025), 1–20.
- [40] Liang Yang, Zhenna Li, Jiaming Zhuo, Jing Liu, Ziyi Ma, Chuan Wang, Zhen Wang, and Xiaochun Cao. 2025. Graph Contrastive Learning with Joint Spectral Augmentation of Attribute and Topology. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Toby Walsh, Julie Shah, and Zico Kolter (Eds.). AAAI Press, 21983–21991. doi:10.1609/AAAI.V39I20.35506
- [41] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph Contrastive Learning with Augmentations. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 5812–5823. https://proceedings.neurips.cc/paper_files/paper/2020/file/3fe230348e9a12c13120749e3f9fa4cd-Paper.pdf
- [42] Si Zhang, Hanghang Tong, Jiejun Xu, and Ross Maciejewski. 2019. Graph convolutional networks: a comprehensive review. *Computational Social Networks* 6, 1 (2019), 1–23.
- [43] Jialun Zheng, Jie Liu, Jiannong Cao, Xiao Wang, Hanchen Yang, Yankai Chen, and Philip S Yu. 2025. DP-DGAD: A generalist dynamic graph anomaly detector with dynamic prototypes. *arXiv preprint arXiv:2508.00664* (2025).

- [44] Jialun Zheng, Divya Saxena, Jiannong Cao, Hanchen Yang, and Penghui Ruan. 2024. Inductive Spatial Temporal Prediction Under Data Drift with Informative Graph Neural Network. In *International Conference on Database Systems for Advanced Applications*. Springer, 169–185.
- [45] Yun Zhu, Haizhou Shi, Zhenshuo Zhang, and Siliang Tang. 2024. MARIO: Model Agnostic Recipe for Improving OOD Generalization of Graph Contrastive Learning. In *Proceedings of the ACM Web Conference* (Singapore, Singapore) (WWW '24). Association for Computing Machinery, New York, NY, USA, 300–311.
- [46] Yanqiao Zhu, Yichen Xu, Feng Yu, Qiang Liu, Shu Wu, and Liang Wang. 2020. Deep graph contrastive representation learning. *arXiv preprint arXiv:2006.04131* (2020).
- [47] Minhao Zou, Yutong Wang, Xiaofeng Meng, Zhongxue Gan, Chun Guan, and Siyang Leng. 2025. Multi-head graph contrastive learning with hop augmentation for node classification. *Pattern Recognition* (2025), 112055.

A Notations

Table 5 introduces the details of notations in SPGCL.

Table 5: Notations.

Notation	Description
Pattern	
$\mathcal{G}$	Input graph
$\mathcal{V}$	The node set of $\mathcal{G}$
$ \mathcal{V} $	The number of nodes in $\mathcal{V}$
$\mathcal{E}$	The edge set of $\mathcal{G}$
$\mathbf{A}$	The adjacency matrix of $\mathcal{G}$
$\mathbf{X}$	The feature matrix of $\mathcal{G}$
$\mathbf{F}$	The dimension of nodes' input feature
$\mathbf{Y}$	The label matrix of $\mathcal{V}$
$\mathbf{V}$	SP node feature
$\mathbf{E}$	SP edge feature
$\phi_x(\cdot)$	Projection function from $\mathbf{V}$ to $\mathbf{X}$
$\phi_y(\cdot)$	Projection function from $\mathbf{V}$ to $\mathbf{Y}$
Augmentation	
$\hat{\mathbf{X}}$	The expended node feature matrix
ϵ	Gaussian blur for feature augmentation
$\mathbf{X}^1, \mathbf{X}^2$	The augmented node feature matrix
$\mathbf{A}^1, \mathbf{A}^2$	The augmented adjacency matrix
p_e	The edge dropping probability
SP-Adj	
$\mathcal{P}$	The set of five subgraph patterns
P_i	The i_{th} subgraph patterns
$\mathbf{A}^{P_i}$	The subgraph pattern adjacency matrix of P_i

B Dataset Statistics

Table 6 provides detailed statistical information for these six datasets. Where "#Node", "#Edge" and "#Class" respectively represent the number of nodes, edges and node label categories. We choose the five most common subgraph patterns in social networks and transportation networks. And the last five rows represents the average number of occurrences of five kinds of subgraph patterns in each dataset.

C Experimental Setup

In the context of unsupervised methods, a linear classifier is employed to predict node labels. The maximum epoch is restricted to 2,000, with a tolerance of 20 in place. 10% of the datasets is reserved for training the classifiers, while the remaining 90% serves as test datasets. In the case of supervised methods, we adhere to the original recommended hyperparameters. For each node type, a

Table 6: Dataset statistics.

Dataset	Acmv9	Dblpv7	Citationv1	Polblogs	Computers	Photo
Node	8,779	5,469	8,769	1,224	13,752	7,650
Edge	13,590	8,098	14,798	16,718	245,861	119,081
Classes	5	5	5	2	10	8
	3,435	1,740	3,666	101,115	17,854,688	717,400
	50,747	41,438	96,486	1,038,396	43,801,479	9,984,393
	608	397	712	422,696	7,764,861	3,330,055
	2,427	2,007	3,756	1,389,540	52,690,704	12,738,922
	5,145	3,173	7,542	1,128,796	35,803,523	7,736,511

training set comprising 20 samples is randomly selected, while 500 samples constitute the validation set, and the rest samples make up the testing set. All experimental computations are carried out using Python 3.10 and the Pytorch framework.

D Theoretical Analysis of the SPGCL loss

We present a mathematical analysis of SPGCL's contrastive loss function, demonstrating that it implements structural gradient re-weighting for hard negative mining. We prove that SPGCL amplifies gradients for structurally similar negative pairs proportionally to their subgraph pattern weights, leading to improved separation in the embedding space.

D.1 Exact SPGCL Formulation from The Paper

The structural weight is defined exactly as in the original paper:

$$w(i, j) = \frac{\mathbf{M}_{SP}(i, j)}{\mathbf{M}_D(i, j)}, \quad \forall i, j \in \mathcal{V} \quad (21)$$

with the convention that $w(i, i) = \mathbf{M}_{SP}(i, i)$ to avoid division by zero when $i = j$.

For node embeddings $\mathbf{u}_i, \mathbf{v}_j \in \mathbb{R}^d$ from two augmented views, define similarity $\delta(i, j) = \mathbf{u}_i^\top \mathbf{v}_j$ (cosine similarity). The SPGCL loss follows Equations (19)-(20) from the original paper.

D.1.1 Original SPGCL Loss Function. For anchor node i , the SPGCL loss is:

$$\mathcal{L}_{SPGCL}(i) = -\log \frac{e^{\delta(i, i)/\tau}}{\mathcal{N}_i} \quad (22)$$

where the denominator $\mathcal{N}_i$ combines both negative terms from the original Equation (20):

$$\mathcal{N}_i = \sum_{k=1}^N \frac{w(i, k) e^{\delta(i, k)/\tau}}{\alpha} + \sum_{k=1, k \neq i}^N \frac{w(i, k) e^{\delta(i, k)/\tau}}{\alpha} \quad (23)$$

Here $\tau > 0$ is temperature and $\alpha > 0$ is a normalization constant.

Remark 1(Normalization Constant). The normalization constant α appears in both summations. In practice, α can be set to $\alpha = 1$ or absorbed into the learning rate, as noted in the original paper. For theoretical analysis, we keep α explicit but note that it cancels in gradient ratios.

D.1.2 Simplified Notation. For clarity, we note that for $k \neq i$, the similarity $\delta(i, k)$ appears in both summations with the same weight $w(i, k)$. Thus:

$$\mathcal{N}_i = \frac{w(i, i)e^{\delta(i, i)/\tau}}{\alpha} + 2 \sum_{k \neq i} \frac{w(i, k)e^{\delta(i, k)/\tau}}{\alpha} \quad (24)$$

This equivalent form simplifies subsequent analysis.

D.2 Exact Gradient Analysis

D.2.1 Gradient Computation. **Lemma 1** (SPGCL Gradient for Negative Pairs). For any anchor node i and negative node $j \neq i$, the gradient of $\mathcal{L}_{\text{SPGCL}}(i)$ with respect to similarity $\delta(i, j)$ is:

$$\frac{\partial \mathcal{L}_{\text{SPGCL}}(i)}{\partial \delta(i, j)} = \frac{2}{\tau} \cdot \frac{w(i, j)e^{\delta(i, j)/\tau}}{\alpha \mathcal{N}_i} \quad (25)$$

From Equation (23), we have:

$$\mathcal{L}_{\text{SPGCL}}(i) = -\frac{\delta(i, i)}{\tau} + \log \mathcal{N}_i \quad (26)$$

Taking the derivative with respect to $\delta(i, j)$ for $j \neq i$:

$$\begin{aligned} \frac{\partial \mathcal{L}_{\text{SPGCL}}(i)}{\partial \delta(i, j)} &= \frac{1}{\mathcal{N}_i} \cdot \frac{\partial \mathcal{N}_i}{\partial \delta(i, j)} \\ &= \frac{1}{\mathcal{N}_i} \cdot \frac{\partial}{\partial \delta(i, j)} \left[\frac{w(i, i)e^{\delta(i, i)/\tau}}{\alpha} + 2 \sum_{k \neq i} \frac{w(i, k)e^{\delta(i, k)/\tau}}{\alpha} \right] \\ &= \frac{1}{\mathcal{N}_i} \cdot \frac{2w(i, j)}{\alpha \tau} e^{\delta(i, j)/\tau} \end{aligned} \quad (27)$$

The factor 2 appears because $\delta(i, j)$ appears in both summations of the original Equation (24) (when $k = j$).

D.2.2 Comparison with Uniform InfoNCE. **Lemma 2** (Uniform InfoNCE Gradient). For the standard InfoNCE loss (uniform weighting, $w(i, j) = 1$ for all i, j), the gradient for negative pair (i, j) is:

$$\frac{\partial \mathcal{L}_{\text{InfoNCE}}(i)}{\partial \delta(i, j)} = \frac{2}{\tau} \cdot \frac{e^{\delta(i, j)/\tau}}{\alpha \mathcal{N}_{\text{unif}, i}} \quad (28)$$

where $\mathcal{N}_{\text{unif}, i} = \frac{e^{\delta(i, i)/\tau}}{\alpha} + 2 \sum_{k \neq i} \frac{e^{\delta(i, k)/\tau}}{\alpha}$.

D.2.3 Gradient Ratio Analysis. **Definition 1** (Gradient Ratio) For anchor i and negative $j \neq i$, define the gradient ratio between SPGCL and uniform InfoNCE:

$$R(i, j) = \frac{\partial \mathcal{L}_{\text{SPGCL}}(i) / \partial \delta(i, j)}{\partial \mathcal{L}_{\text{InfoNCE}}(i) / \partial \delta(i, j)} \quad (29)$$

Theorem 1 (Exact Gradient Ratio) The gradient ratio $R(i, j)$ has the exact expression:

$$R(i, j) = w(i, j) \cdot \frac{\mathcal{N}_{\text{unif}, i}}{\mathcal{N}_i} \quad (30)$$

From Lemmas 1 and 2:

$$R(i, j) = \frac{\frac{2}{\tau} \cdot \frac{w(i, j)e^{\delta(i, j)/\tau}}{\alpha \mathcal{N}_i}}{\frac{2}{\tau} \cdot \frac{e^{\delta(i, j)/\tau}}{\alpha \mathcal{N}_{\text{unif}, i}}} = w(i, j) \cdot \frac{\mathcal{N}_{\text{unif}, i}}{\mathcal{N}_i} \quad (31)$$

D.2.4 Bounds on Gradient Ratio. **Theorem 2** (Bounds on Gradient Ratio) Let $w_{\min}^{(i)} = \min_{k \neq i} w(i, k)$ and $w_{\max}^{(i)} = \max_{k \neq i} w(i, k)$. Then for any negative $j \neq i$:

$$\frac{w(i, j)}{w_{\max}^{(i)}} \cdot \frac{1}{1 + \frac{w(i, i)e^{\delta(i, i)/\tau}}{2w_{\max}^{(i)} \sum_{k \neq i} e^{\delta(i, k)/\tau}}} \leq R(i, j) \leq \frac{w(i, j)}{w_{\min}^{(i)}} \cdot \frac{1}{1 + \frac{w(i, i)e^{\delta(i, i)/\tau}}{2w_{\min}^{(i)} \sum_{k \neq i} e^{\delta(i, k)/\tau}}} \quad (32)$$

We bound the denominator $\mathcal{N}_i$:

$$\mathcal{N}_i = \frac{w(i, i)e^{\delta(i, i)/\tau}}{\alpha} + \frac{2}{\alpha} \sum_{k \neq i} w(i, k)e^{\delta(i, k)/\tau} \quad (33)$$

For the upper bound on $R(i, j)$, we need a lower bound on $\mathcal{N}_i$ and an upper bound on $\mathcal{N}_{\text{unif}, i}$. First:

$$\mathcal{N}_i \geq \frac{w(i, i)e^{\delta(i, i)/\tau}}{\alpha} + \frac{2w_{\min}^{(i)}}{\alpha} \sum_{k \neq i} e^{\delta(i, k)/\tau} \quad (34)$$

$$\mathcal{N}_{\text{unif}, i} = \frac{e^{\delta(i, i)/\tau}}{\alpha} + \frac{2}{\alpha} \sum_{k \neq i} e^{\delta(i, k)/\tau} \quad (35)$$

Thus:

$$\frac{\mathcal{N}_{\text{unif}, i}}{\mathcal{N}_i} \leq \frac{\frac{e^{\delta(i, i)/\tau}}{\alpha} + \frac{2}{\alpha} \sum_{k \neq i} e^{\delta(i, k)/\tau}}{\frac{w(i, i)e^{\delta(i, i)/\tau}}{\alpha} + \frac{2w_{\min}^{(i)}}{\alpha} \sum_{k \neq i} e^{\delta(i, k)/\tau}} \quad (36)$$

Dividing numerator and denominator by $\frac{2}{\alpha} \sum_{k \neq i} e^{\delta(i, k)/\tau}$:

$$\frac{\mathcal{N}_{\text{unif}, i}}{\mathcal{N}_i} \leq \frac{\frac{e^{\delta(i, i)/\tau}}{2 \sum_{k \neq i} e^{\delta(i, k)/\tau}} + 1}{\frac{w(i, i)e^{\delta(i, i)/\tau}}{2 \sum_{k \neq i} e^{\delta(i, k)/\tau}} + w_{\min}^{(i)}} = \frac{1 + A}{w(i, i)A + w_{\min}^{(i)}} \quad (37)$$

where $A = \frac{e^{\delta(i, i)/\tau}}{2 \sum_{k \neq i} e^{\delta(i, k)/\tau}}$.

Multiplying by $w(i, j)$ gives the upper bound. The lower bound follows similarly by using $w_{\max}^{(i)}$ instead of $w_{\min}^{(i)}$.

Remark 2 (Tightness of Bounds) The bounds become tighter when $A = \frac{e^{\delta(i, i)/\tau}}{2 \sum_{k \neq i} e^{\delta(i, k)/\tau}}$ is small, which typically holds in contrastive learning as positive pairs are much rarer than negatives.

In practice, $A \ll 1$ and the bounds simplify to $\frac{w(i, j)}{w_{\max}^{(i)}} \lesssim R(i, j) \lesssim \frac{w(i, j)}{w_{\min}^{(i)}}$.

D.2.5 Structural Hard Negatives. **Definition** (Structural Hard Negative) For anchor i , define structural hard negatives as nodes with above-median structural weight:

$$\mathcal{H}_i = \{j \neq i \mid w(i, j) > \text{median}_{k \neq i} w(i, k)\} \quad (38)$$

Corollary 1 (Gradient Amplification for Hard Negatives) Assuming $A \ll 1$ and $w(i, i) \approx 0$, for $j \in \mathcal{H}_i$ we have:

$$R(i, j) \approx \frac{w(i, j)}{\mathbb{E}_{k \neq i}[w(i, k)]} > 1 \quad (39)$$

Thus, hard negatives ($w(i, j) > \mathbb{E}[w(i, \cdot)]$) receive amplified gradients relative to uniform InfoNCE.

D.3 Information-Theoretic Interpretation

D.3.1 Weighted InfoNCE Formulation. **Theorem 3**(SPGCL as Weighted InfoNCE) The SPGCL loss can be expressed as:

$$\mathcal{L}_{\text{SPGCL}} = \mathbb{E}_{i \sim \mathcal{V}} \left[-\log \frac{e^{\delta(i,i)/\tau}}{\mathbb{E}_{k \sim \mathcal{V} \setminus \{i\}} [w(i,k)e^{\delta(i,k)/\tau}]} \right] + \log 2 + \log \mathbb{E}[w(i, \cdot)] + \log \alpha \quad (40)$$

where the expectation over k is uniform over all nodes except i .

From Equation (24), we have:

$$\mathcal{N}_i = \frac{w(i,i)e^{\delta(i,i)/\tau}}{\alpha} + 2 \sum_{k \neq i} \frac{w(i,k)e^{\delta(i,k)/\tau}}{\alpha} \quad (41)$$

Taking the empirical expectation over negatives:

$$\frac{2}{N-1} \sum_{k \neq i} w(i,k)e^{\delta(i,k)/\tau} = 2 \cdot \mathbb{E}_{k \sim \mathcal{V} \setminus \{i\}} [w(i,k)e^{\delta(i,k)/\tau}] \quad (42)$$

Thus:

$$\mathcal{N}_i = \frac{w(i,i)e^{\delta(i,i)/\tau}}{\alpha} + (N-1)\alpha \cdot \mathbb{E}_k [w(i,k)e^{\delta(i,k)/\tau}] \quad (43)$$

Setting $\alpha = \frac{1}{N-1}$ and neglecting the typically small term $w(i,i)e^{\delta(i,i)/\tau}$, we obtain:

$$\mathcal{N}_i \approx 2\mathbb{E}_k [w(i,k)e^{\delta(i,k)/\tau}] \quad (44)$$

Substituting into the loss and taking expectations yields the result.

D.3.2 Connection to Mutual Information. **Remark 3**((Weighted Mutual Information Bound) When the encoder approaches optimality such that $e^{\delta(i,j)/\tau} \propto \frac{p(j|i)}{p(j)}$, SPGCL maximizes a lower bound on the weighted mutual information:

$$I_w(U; V) = I(U; V) + \mathbb{E}_{i,j} [\log w(i, j)] - \log \mathbb{E}[w(i, j)] \quad (45)$$

where $I(U; V)$ is the standard mutual information between views.

E Time Complexity Analysis of SP Adjacency Matrix

We define $n = |V|$ as number of nodes, $m = |E|$ as number of edges, and $d_{\max}$ as maximum degree.

E.1 Edge Subgraph Adjacency Matrix

The algorithm is shown as Algorithm 1. Time complexity is $O(n + m)$.

E.2 Triangle Subgraph Adjacency Matrix

The algorithm is shown as Algorithm 2. The Degree sorting is $O(n \log n)$. Each triangle counted once. And the worst case is $O(m^{3/2})$.

E.3 Wedge Subgraph Adjacency Matrix

The algorithm is shown as Algorithm 3. Traversing all nodes costs $O(n)$. Checking neighbor pairs costs $O(d_{\max}^2)$. Thus, the total time complexity is $O(n \cdot d_{\max}^2)$.

E.4 Quadrangle Subgraph Adjacency Matrix

Time complexity is $O(n \cdot d_{\max}^2 + m^{3/2})$. Degree sorting is $O(n \log n)$. Finding 2-hop neighbors costs $O(d(v_i) \cdot d_{\max})$. Checking pairs costs $O(\text{candidates}^2)$, worst $O(d_{\max}^2)$. Thus the total time complexity is $O(n \cdot d_{\max}^2 + m^{3/2})$.

Algorithm 1 Construct Edge Subgraph Adjacency Matrix

```

1: for each node  $v \in V$  do
2:   for each outgoing neighbor  $u$  of  $v$  do
3:     {Edge  $(v, u)$  contributes to edge subgraph pattern}
4:      $M[v][u] \leftarrow M[v][u] + 1$  {Increment count for directed edge  $v \rightarrow u$ }
5:     if graph is undirected (symmetrization required) then
6:        $M[u][v] \leftarrow M[u][v] + 1$  {Symmetric update for undirected edge  $u \leftrightarrow v$ }
7:     end if
8:   end for
9: end for
10: {Edge subgraph adjacency matrix now contains counts of edge co-occurrences}
11: return  $M$ 

```

Algorithm 2 Construct Triangle Subgraph Adjacency Matrix

```

1: Sort nodes by degree to obtain order
2: for  $v \in V$  do
3:    $S \leftarrow \{u \in N(v) \mid \text{order}[u] > \text{order}[v]\}$ 
4:   for  $(u_1, u_2)$  in unordered pairs of  $S$  do
5:     if edge  $(u_1, u_2)$  exists then
6:        $M[v][u_1] \leftarrow M[v][u_1] + 1$ 
7:        $M[v][u_2] \leftarrow M[v][u_2] + 1$ 
8:        $M[u_1][u_2] \leftarrow M[u_1][u_2] + 1$ 
9:     end if
10:   end for
11: end for

```

Algorithm 3 Construct Wedge Subgraph Adjacency Matrix

```

1: for each node  $center$  do
2:    $N \leftarrow$  all neighbors of  $center$ 
3:   for each pair  $(u_1, u_2) \in N \times N$  do
4:     if subgraph  $\{center, u_1, u_2\}$  matches target motif then
5:       Update all pair weights
6:     end if
7:   end for
8: end for

```

F Subgraph Pattern Adjacency Composition Analysis

We conducted node classification experiments on Polblogs with different subgraph pattern combinations. We observe that P_1, P_2 and P_5 exhibit the highest contributions to the model, which may be attributed to the citation network being rich in subgraph patterns of these types. The combination of P_1, P_2 , and P_5 achieved an average accuracy exceeding 0.94. In addition, the combinations of low-order subgraph patterns significantly outperform those of high-order subgraph patterns. We postulate two underlying reasons: First, low-order subgraphs exhibit higher occurrence frequencies than high-order subgraphs, rendering them less susceptible to external noise and endowing them with stronger generalization capabilities. Second, high-order subgraphs can be further composed and interpreted by low-order subgraphs.